CSC 453

Spring 2026 - Day 16

Admin

- Lab due Thursday
- Assignment 4 due Wednesday
- Quiz due next Monday
- Reminder the final is on Tuesday, June 9th at 13:10 in the lab room

File system consistency

Questions to consider

- What can go wrong when a crash interrupts a multi-step file system operation like appending to a file?
- How does `fsck` detect and repair file system inconsistencies, and what are its limitations?
- How does journaling maintain consistency across crashes, and what ordering guarantees matter?
- What is the difference between ordered and journal modes?

Consistency

- Machines can crash / reboot unexpectedly.
- Example: bitmap + inode + data block. Append operation results in three writes
  - → What outcomes are possible from a crash?

Consistency (cont'd)

- One operation succeeds
  - → Just inode is updated
    - Pointing to garbage
    - Inconsistent with bitmap
  - → Just the bitmap is updated
    - Inconsistent with inode
    - If we do nothing, we have a space leak
  - → Just the data block is written (no inconsistency for FS, still bad)

Consistency (cont'd)

- Two operations succeed
  - → Inode and data, not bitmap
    - Inconsistent FS
  - → Bitmap and data, not inode
    - Inconsistent and we have no idea who (inode) owns the data
  - → Bitmap + inode done, not data
    - Big problem. FS is consistent, but data is garbage

Consistency (cont'd)

- What should we do?
- Atomicity across multiple disk writes isn't guaranteed: a crash between writes leaves the FS in a partial state
- We can allow things to fail, and try to recover from inconsistencies after the fact
- We can use a journal

Consistency checking

- `fsck`: file system consistency check. Uses up to six passes:
  - → Pass 1: scan superblock and all inodes
    - for each inode, ensure metadata makes sense (valid permissions, sensible/valid block numbers)
  - → Pass 2 & 3: check directories
    - Do `.` and `..` create a cycle-less namespace?
    - Orphaned directories are placed in `lost+found`
  - → Pass 4: Check ref counts
  - → Pass 5: check bitmaps
    - Do the in-use bitmaps match the current inode, block state?
  - → Pass 6: check sectors (optional)

Consistency checking (cont'd)

- `fsck` is a powerful tool, but
  - → It requires the file system to be unmounted, which can lead to downtime
  - → This is a massive operation, can take hours… Should we really take this extreme approach for a potentially small issue?
  - → It doesn't fix all problems (bitmap and inode are consistent but point to garbage)
  - → Allows things to break then tries to repair
- There has to be a better way to maintain consistency in the first place, which leads us to journaling

Journaling

- Like any hard systems problem, we'll steal ideas from others (in this case databases)
- The idea is to write an operation's intent to a journal before performing the actual operation
- If the system crashes, we can replay the journal to complete any operations that were in-flight
- Journaling can be implemented in different ways:
  - → **Ordered mode**: only journal metadata changes (e.g., inode updates, bitmap changes)
  - → **Journal mode**: both metadata and data are written to the journal

Example: ext3

- Ext2 + journaling
  - → Organized into block groups
  - → Groups sized equal to one or more disk tracks
  - → Each group contains
    - a copy of the superblock (although not all are current)
    - a group description (block group metadata)
      - block number of blocks; allocation bitmap
      - block number of inodes; allocation bitmap
      - number of free blocks; inodes and directories in group
    - Inodes
    - Data blocks

Example: ext3 (cont'd)

- ext3 inodes use a multi-level index
  - → 12 direct, 1 single indirect, 1 double indirect and 1 triple indirect
    - 4KB blocks? 48K, 4MB, 4GB, 4TB
- Attempts to allocate related blocks near each other, near their inodes (within a group): within a cylinder group or in nearby cylinder groups
  - → All blocks of a file accessed in one track access (ideally)
- Fixed number of inodes per group

Ext3 journaling

- Transaction begin and end markers in the journal, plus actual data to be written
- Naive order of operations:
  - → Write journal entry
  - → Write to disk
- What happens if the system crashes in the middle of the journal write (say, Db is not written)?
  - → Could write garbage when replaying the transaction upon boot
  - → Now what?

Ext3 journaling (cont'd)

- Ext3 uses ordered mode journaling, which means that only metadata changes are journaled, and data blocks are written directly to their final location on disk
  - → We have to be *careful* about the order of operations to maintain consistency
- We need to break the transaction into multiple steps, and write them in a specific order to ensure consistency
  - → Write data blocks to their **final disk location** (never to the journal)
  - → Write metadata blocks to the journal
  - → Write transaction end marker to journal (MUST be last, known as a transaction **commit**)
  - → Write (checkpoint) metadata blocks to their final disk location
- The key invariant: data is on disk *before* the journal commit that makes the new metadata visible

What isn’t clear?

Comments? Thoughts?

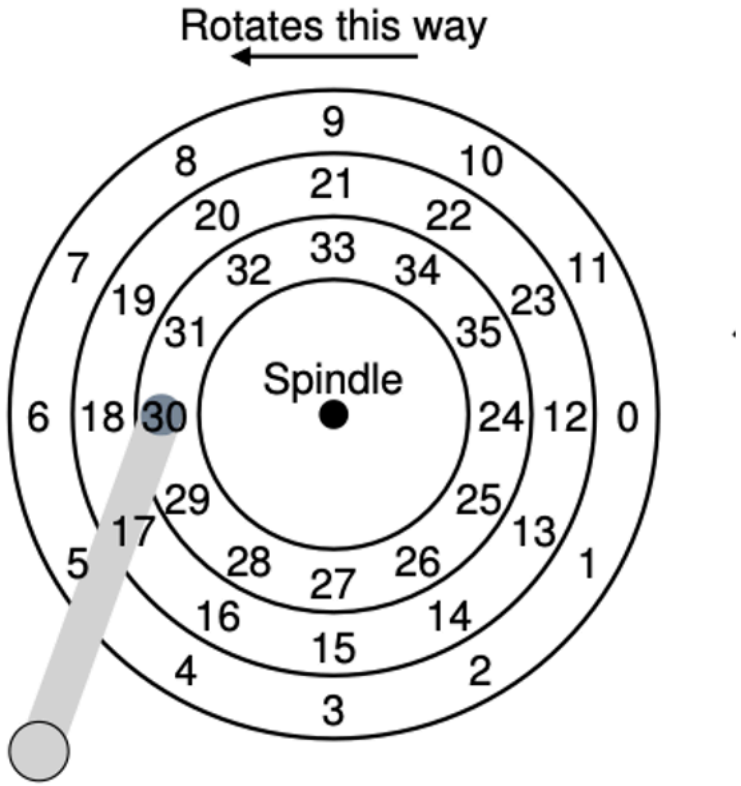
Storage basics

Questions to consider

- What are the three components of HDD access time, and which dominates?
- How does RPM affect average rotational latency?

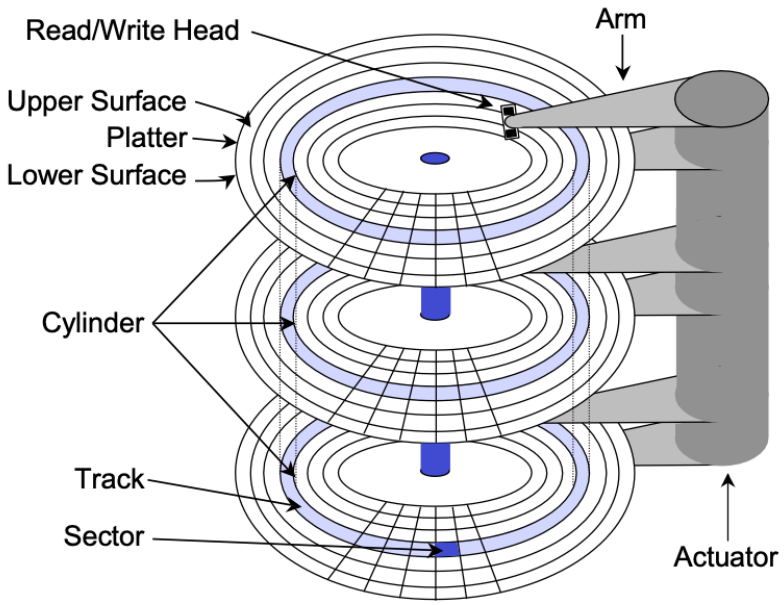
HDDs

- Yes, they're still important / around
- Far more cost effective than SSDs
- Sector: (512 bytes) smallest amount of IO you can do to an HDD
- Tracks: 256KB-1MB in size



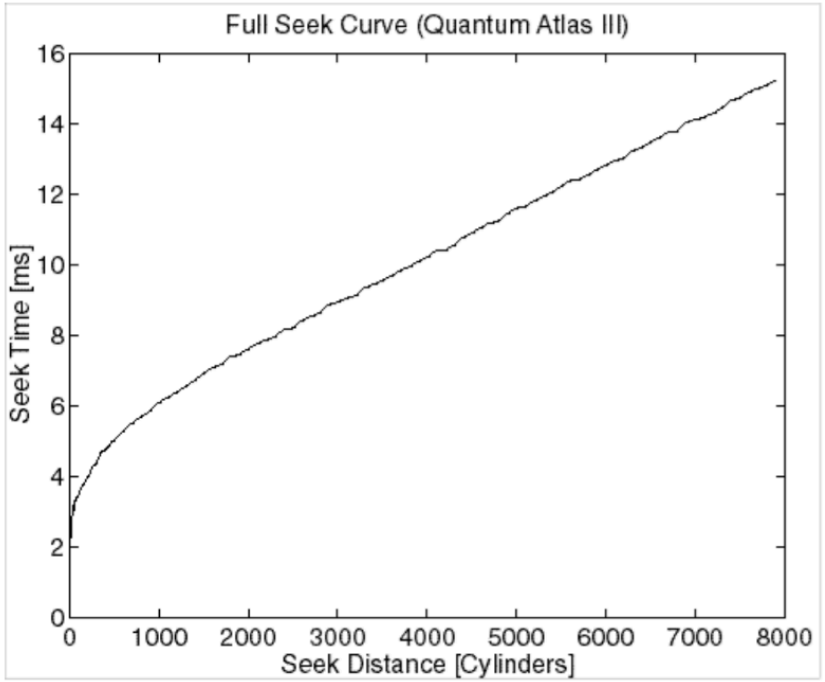
HDDs (cont'd)

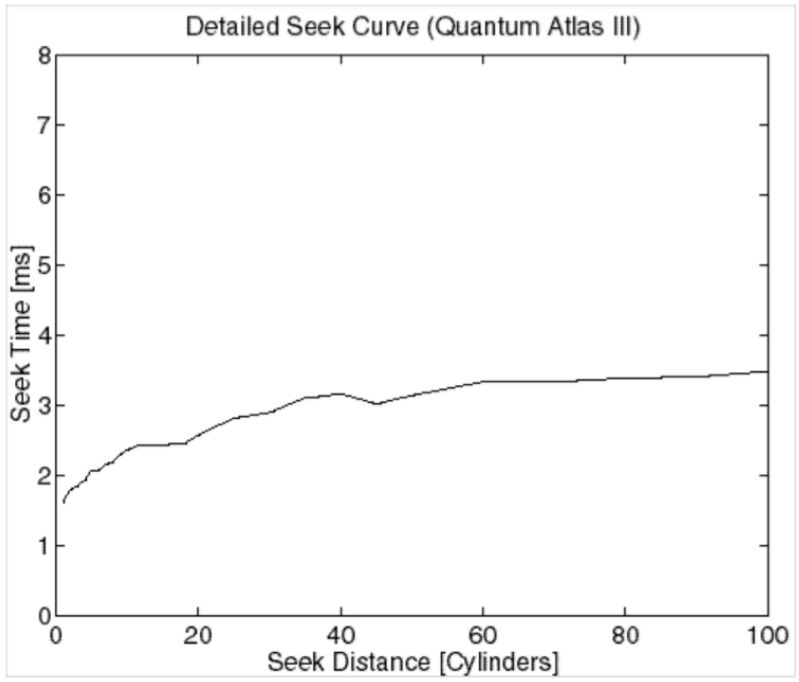
- Actual drives have many platters, are double-sided
- Millions of tracks
- Performance factors?
  - → Seek (moving the arm to position the head over the correct sector)
  - → Rotation (RPMs)
  - → Transfer (time to read/write from the head)
- Which do you think is most burdensome / costly?



Seek time

- Seek: accelerate, coast, settle





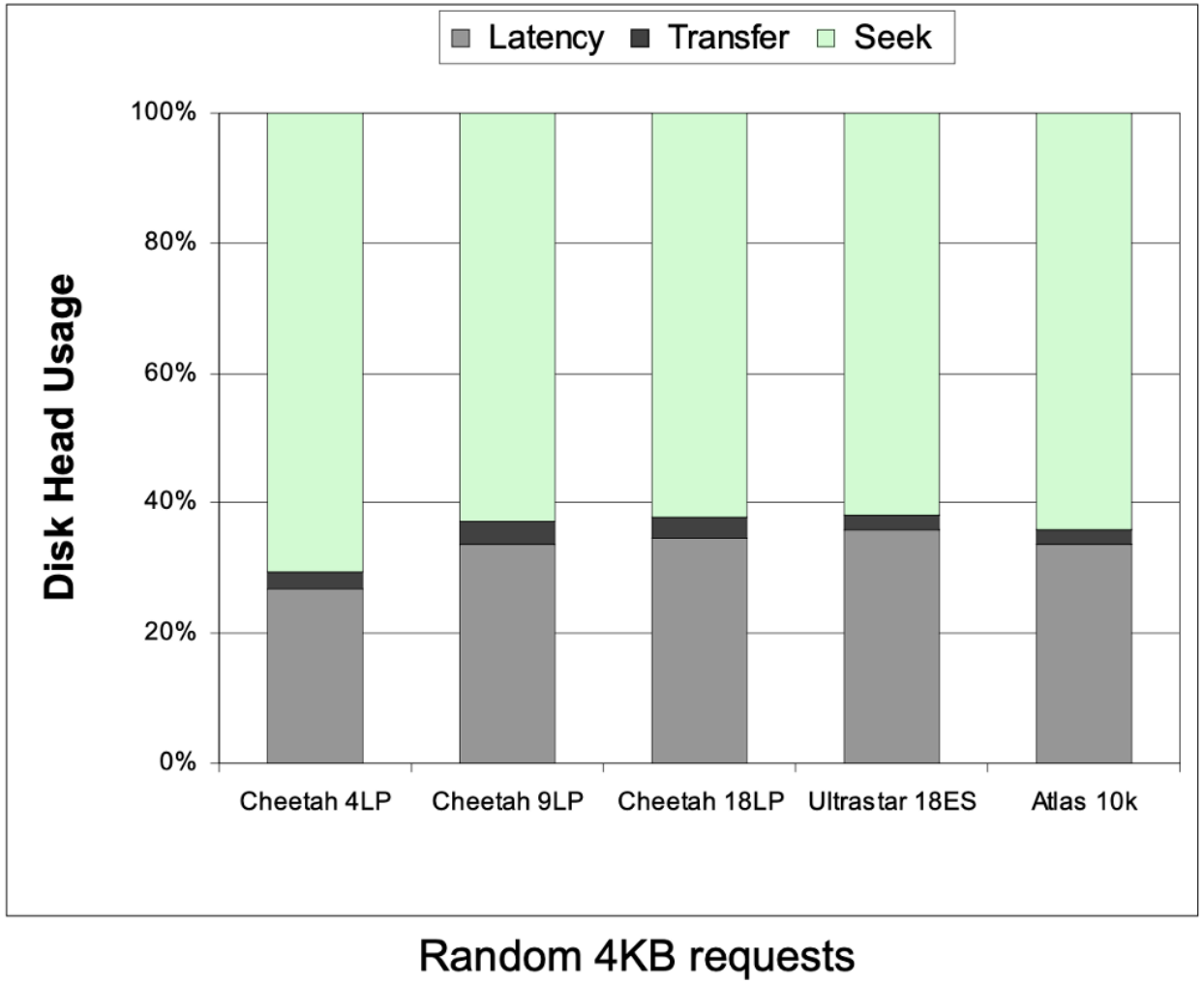
Rotation time

- a function of RPM; average rot. lat. ½ revolution
  - → e.g. 7200RPM = 8.33ms per rotation
    - avg = 4.16ms
- Common HDD speeds:
  - → 5400 (yikes, good for cold storage)
  - → 7200 (decent middle ground)
  - → 10K, 15K (good for a lot of random accesses); use case?
    - DBs

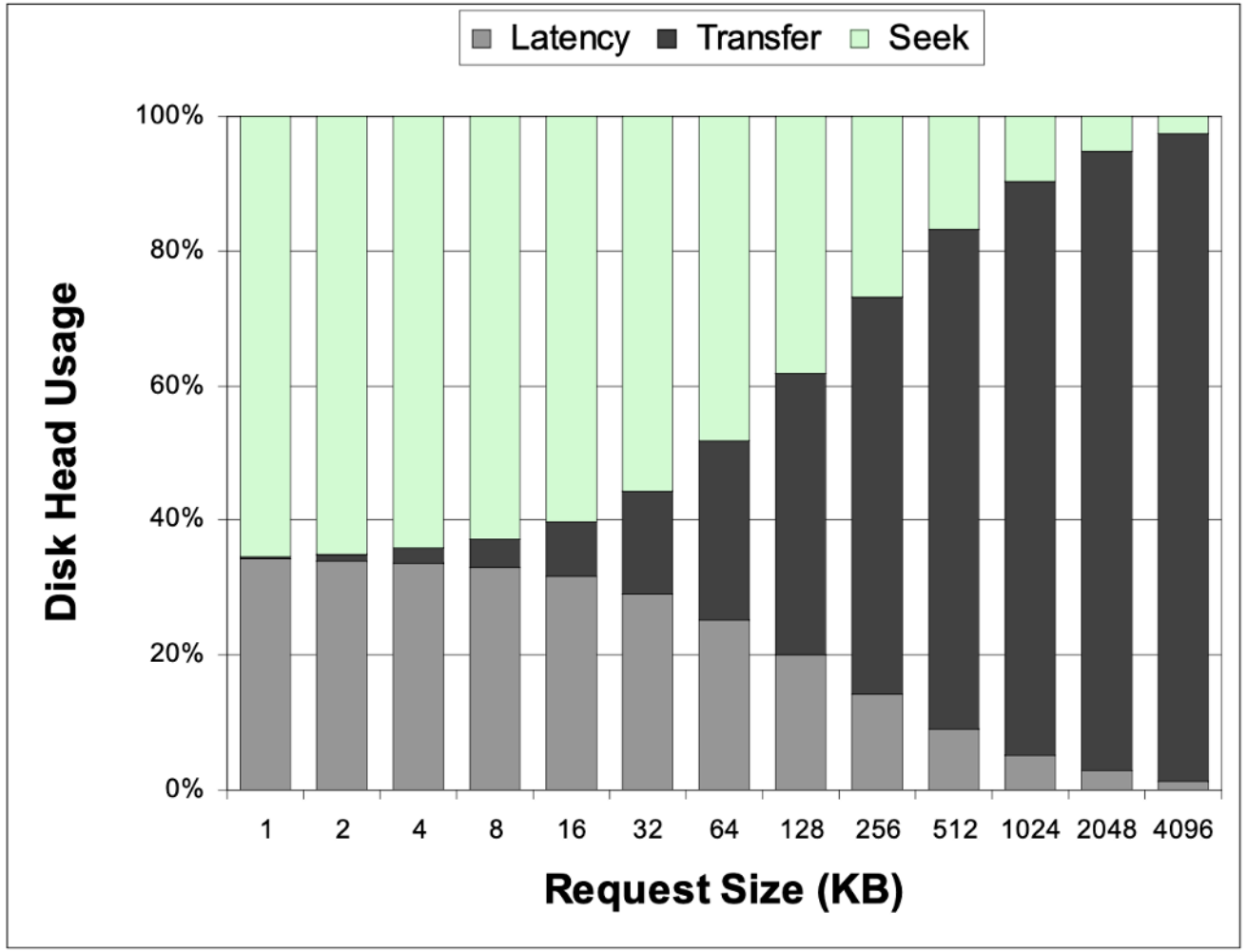
Transfer time

- simple case (all data in one track):
  - → `sectors desired * time for revolution / sectors per track`
- complex case (mult. tracks): above + ? (seeks, track & cyl. switches)

Where does the time go?



Where does the time go? It depends.



What isn’t clear?

Comments? Thoughts?

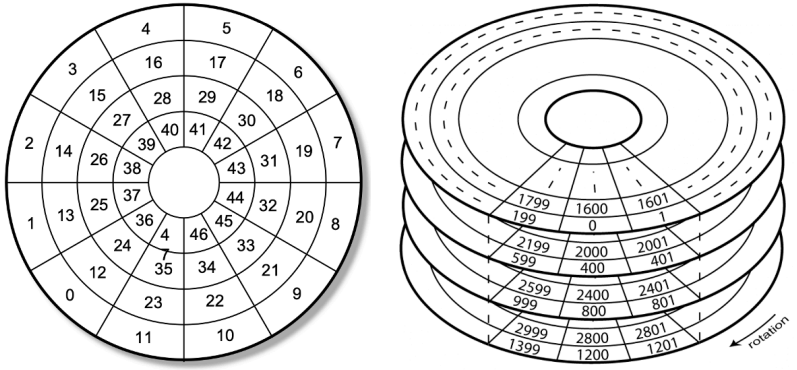
HDD organization

Questions to consider

- What is track skew, and why does it improve sequential read performance across cylinders?
- Why do outer tracks hold more data than inner tracks on a constant-RPM drive, and how does zoning address this?
- When a sector goes bad, what are the two ways a drive can handle it, and what is the trade-off?

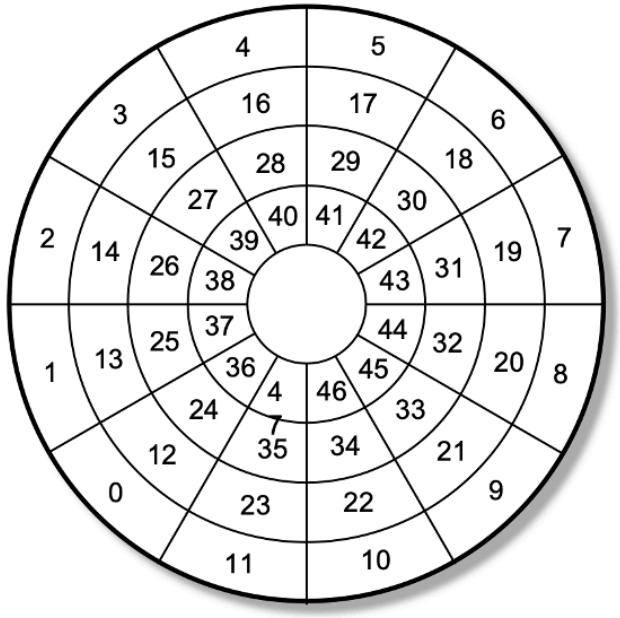
Simple organization (cont'd)

- What's presented by the disk drive: a linear array of sectors
  - → This is a logical abstraction
- Drive must map logical blocks to physical blocks



Complication: skew

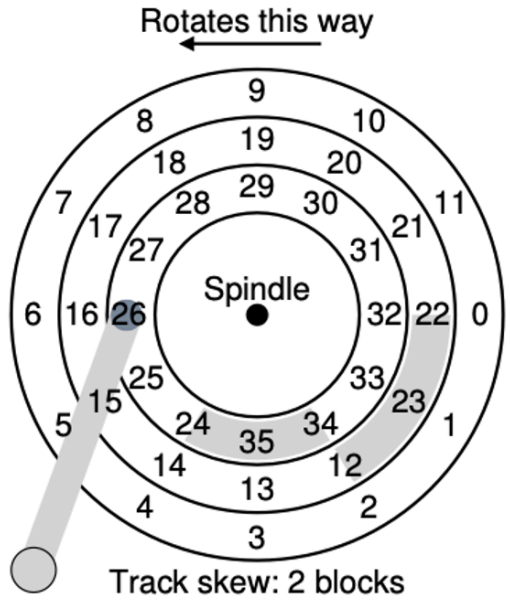
- Sequential reads / write are ideal
- Unfortunately, seeks exist, and sector sequences can cross cylinders / tracks
- E.g., if you are reading sectors 9-12 on the disk you need to seek to a new cylinder
  - → What downside does this present?
    - By the time you seek, sector 12 has already spun past the head, you need to wait for it to rotate back around



Complication: skew (cont'd)

- What should we do?
- Reorganize where sectors are located on the physical device

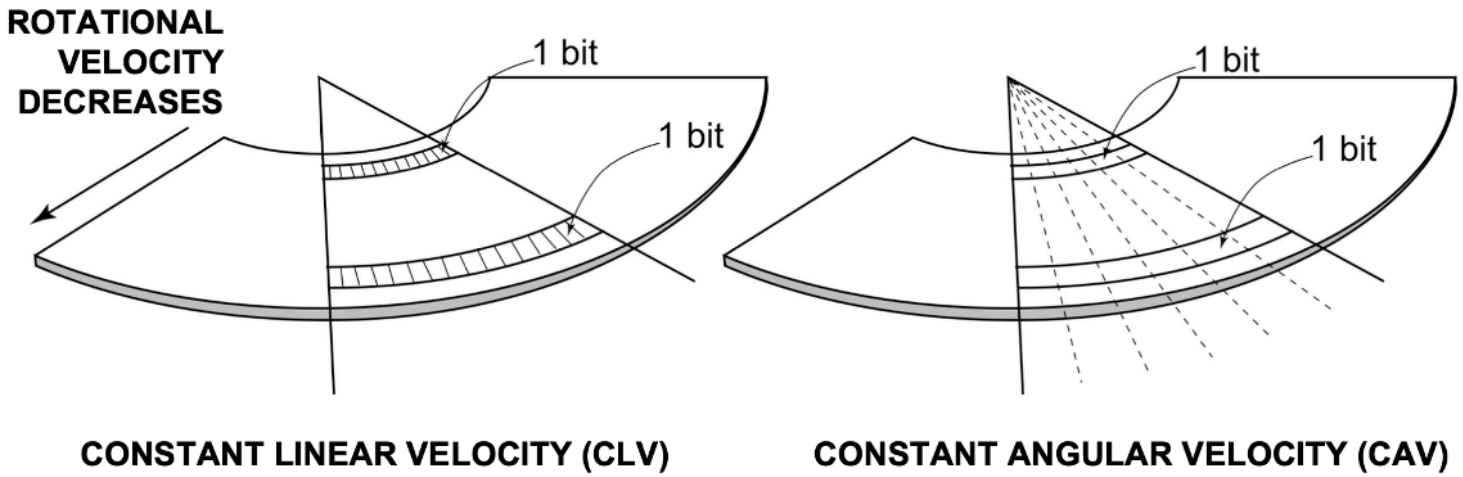
  → Requires tuning based on the specific device characteristics, in this example it's 2 blocks off (to go from sector 23 to sector 24)



Complication: zones

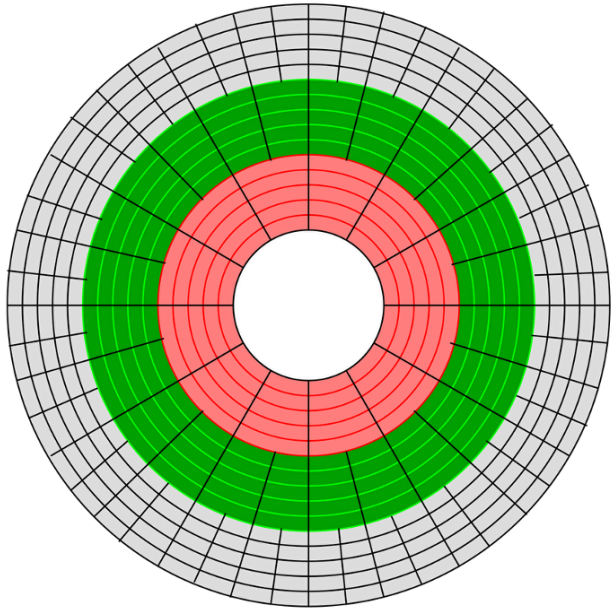
- Linear velocity vs. angular velocity
- We talk about drives in terms of constant RPM: what is the consequence of this for tracks?

  → Outer tracks have more space to put data



Complication: zones (cont'd)

- What should we do?
- We could slow the rotation on outer tracks
  - → Some older systems did this
- Option 2: (common)
  - → Place more sectors per zone on outer tracks
  - → Complicates logic



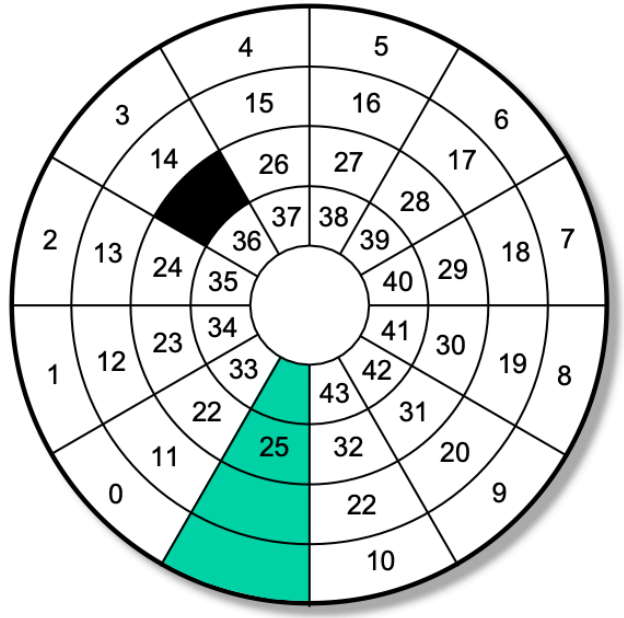
Complication: defects

- Over time, portions of media become unusable (e.g. fail to hold charge, exhibit many errors);
- How should we handle a defect?

  → We keep additional physical space for “spares” (both tracks and sectors)

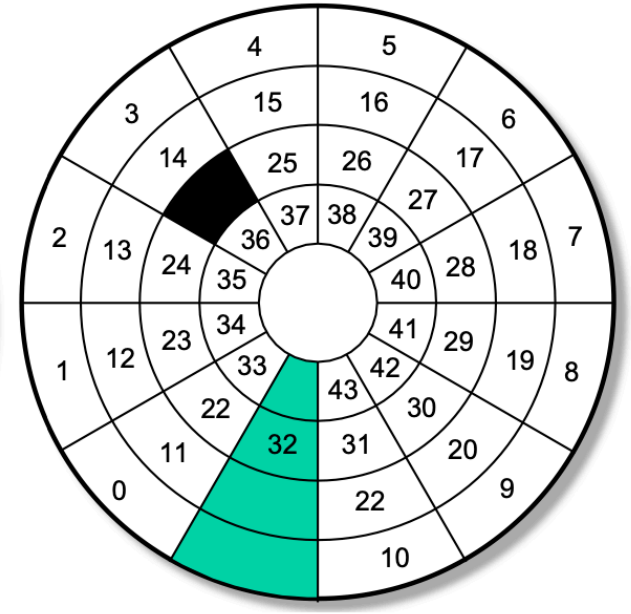
Complication: defects (cont'd)

- How should we handle a defect?
- **Remapping**: move the sector to a location in the reserved spare pool
- Ongoing process, taken care of by drive firmware
- Do you see any downsides?
  - → Sequential access is now broken



Complication: defects (cont'd)

- What else could we do?
  - → Move everything to maintain sequential sectors (known as slipping)
    - This really only works on a fresh drive formatting: hard to do live
  - → Other approaches?
    - Error Correcting Codes (ECC)
      - If you're curious most HDDs use Reed-Solomon



HDD organization summary

- Luckily, the OS no longer needs to care about most of these details: taken care of by firmware on the drive itself
- We'll return to some gory details about disks in a bit (RAID), but first, disk scheduling

What isn’t clear?

Comments? Thoughts?